HawkScan (DAST) — Helix Universal integration

Governed by **§11.4.184(I)** of the constitution. Consumed BY REFERENCE from any project that inherits this submodule (§11.4.28 / §11.4.80-style) — never copied per project.

HawkScan is StackHawk's dynamic application security testing (DAST) scanner. It is a **SaaS-backed commercial tool**, not a self-contained open-source project — the scanner engine (`stackhawk/hawkscan` container image) always calls out to StackHawk's own backend to authenticate and report results. There is no fully-local, no-account mode. This is why it is scaffolded here as a **local runner against a pulled image**, not a git submodule: nothing about HawkScan's own distribution is a clonable source repository, and its `helix-deps.yaml` (§11.4.31) schema is for git-clonable dependencies only.

Files in this directory

File	Purpose
`hawkscan_lib.sh`	Shared functions (sourced, never executed directly).
`hawks-can_install_check.sh`	Verifies the container runtime + image; reports (never fails the build over) credential state.
`hawkscan_run_scan.sh`	Runs a real scan, or fails OPEN with a loud warning if credentials/config are incomplete.
`stack-hawk.yml.example`	Per-project config template — copy to `<project-root>/stackhawk.yml` and fill in.

Step-by-step: obtaining a free-tier StackHawk API key

These steps are **operator-only, one time, per StackHawk account** — an interactive web signup with email verification, which cannot be automated by any script or agent:

1. Go to https://app.stackhawk.com and sign up for the free tier (email + password, a couple of minutes; no credit card required for the free tier at the time of writing).
2. Once logged in, go to **Settings** → **API Keys** in the StackHawk dashboard and generate an API key. Copy it somewhere safe — it is shown once.
3. Export it as an environment variable in your shell session or `.env` file — **never commit it** (§11.4.10):

```
export HAWK_API_KEY="<your key>"
```

Step-by-step: creating a per-project Application

Every project scanned needs its own StackHawk **Application**, which is a **separate manual step from the API key** (also operator-only, also cannot be automated):

1. In the StackHawk dashboard, click **Applications** → **Add Application**.
2. Give it a name (e.g. your project's name) and select an environment (e.g. "Development").
3. Save it — StackHawk issues an `applicationId` (a UUID).
4. Copy `stackhawk.yml.example` from this directory to `<project-root>/stackhawk.yml` and paste the real `applicationId` in, replacing `REPLACE_WITH_REAL_STACKHAWK_APPLICATION_ID`. Set `app.host` to your project's real local target.

Running a scan

```
# From the consuming project, with HAWK_API_KEY exported and
# <project-root>/stackhawk.yml configured:
bash <constitution>/scripts/hawkscan/hawkscan_install_check.sh
bash <constitution>/scripts/hawkscan/hawkscan_run_scan.sh

# Against a different live target than the one in stackhawk.yml:
HAWKSCAN_TARGET_OVERRIDE=http://127.0.0.1:8099 \
  bash <constitution>/scripts/hawkscan/hawkscan_run_scan.sh
```

What happens with no key configured

Per §11.4.184(I), a missing third-party credential is an **operator setup gap, never a build or gate failure**. `hawkscan_run_scan.sh` fails OPEN: with no `HAWK_API_KEY` set, or the placeholder `applicationId` still in `stackhawk.yml`, it prints a clear, loud, non-blocking warning and exits 0. It never blocks other work, and it never fabricates a PASS — see the "Honest boundary" clause of §11.4.184(I): this tooling's compliance bar is "correctly scaffolded with a fail-open guard," never "an account exists" or "a scan ran," until an operator completes the two manual steps above.